THORNS

Technical Design Document

Computer Games Development | CW208 | Year IV

Student Name	Edward Norton
Student Number	C00286807
Supervisor	Omer Ali
Project Head	Martin Harrigan
Institution	SETU Carlow
Date of Submission	13th April 2026
Academic Year	2025-2026

Contents

1. Introduction 4

2. Technology Stack 4

3. Architecture Overview 4

3.1 Design Philosophy 4

3.2 High-Level Architecture Diagram 5

4. Interface Design 6

5. Component Design 7

5.1 Component Overview 7

5.2 SpriteComponent 7

5.3 StatComponent 8

5.4 HUDComponent 8

5.5 ButtonComponent 8

5.6 CursorComponent 8

5.7 DayTimerComponent 8

6. State Management: Pushdown Automaton 8

7. Input System: Command Pattern 9

8. Collision System 10

8.1 Overview 10

8.2 Collision Shape Types 10

8.3 Flyweight Pattern: WorldObjectTemplateManager 10

8.4 SAT Collision 11

8.5 Coordinate Space 11

9. Procedural Map Generation Pipeline 11

9.1 Overview 11

9.2 Phase 1: Voronoi Diagram with Poisson Disk Sampling 12

9.3 Phase 2: Perlin Noise Object Placement 12

9.4 Generation Settings 12

10. Rendering Pipeline 12

10.1 View Separation 12

10.2 Frustum Culling 12

10.3 Shared Texture Atlas 13

11. Enemy System: Object Pool and Manager 13

11.1 Pattern: Object Pool 13

11.2 SavageEnemy 13
11.3 ChomperEnemy 13
12. Performance Analysis 14
12.1 Baseline: Unoptimized Generation 14
12.2 Optimised: Spatial Partition + Poisson Disk Sampling 14
12.3 Regeneration Performance 15
12.4 CPU Profiling: VS Diagnostics 15
12.5 Large Map Stress Test 16
13. Key Class Relationships 16
14. C++17 Features 17
15. File and Module Structure 18
References 19

1. Introduction

This document describes the technical design decisions, software architecture, and implementation patterns used in Thorns, a top-down 2D survival game developed in C++17 using SFML 3.0. The purpose of this document is to communicate, clearly and completely, the internal structure of the system to academic reviewers, future developers, and as a reference during development.

All design decisions are grounded in the Game Programming Patterns catalogue (Nystrom, 2014). Where a pattern has been applied, the relevant section is identified. The document covers architecture, component design, generation algorithms, collision systems, and performance profiling.

2. Technology Stack

Technology	Role	Key Features Used
C++17	Primary language	RAII, smart pointers, std::variant, if constexpr, structured bindings
SFML 3.0	Graphics, windowing, input, audio	Render targets, Views, Sprite, Texture, RenderWindow
tmxlite	TMX map file parsing	Collision shape extraction from Tiled map files
CMake	Build system	Cross-platform project configuration
STL	Data structures and algorithms	std::vector, std::unordered_map, std::unique_ptr, std::variant
Visual Studio (MSVC)	IDE and compiler	Diagnostics profiler used for performance analysis

3. Architecture Overview

3.1 Design Philosophy

Thorns is built on a composition-over-inheritance philosophy. Game entities are assembled from small, single-responsibility components rather than deep inheritance hierarchies. Four thin interface classes (IRenderable, IUpdatable, IPositionable, ICollidable) define contracts that components implement independently.

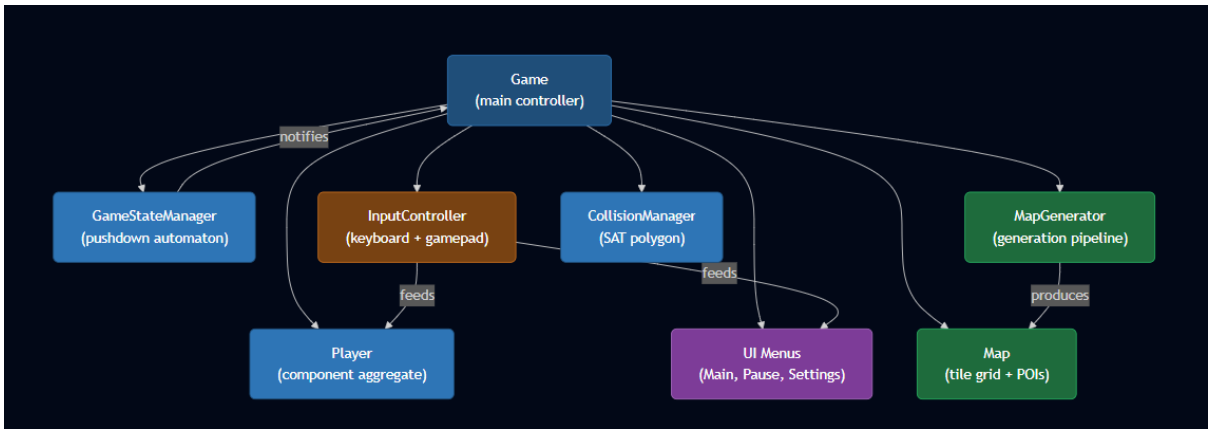
All patterns applied in the project are drawn from Nystrom (2014): Game Programming Patterns. The following patterns are actively used:

Pattern	Where Used	Rationale
Component	Player, WorldObject	Assembles behaviour from small, reusable parts without deep inheritance.

State / Pushdown Automaton	GameStateManager	Stack-based state transitions preserve history (pause - resume).
Command	InputController	Decouples input device from game actions, supports runtime rebinding.
Flyweight	WorldObjectTemplateManager	One collision shape list per object type shared by all instances.
Spatial Partition	SpatialGrid (Voronoi)	O(1) nearest-site lookup by dividing world space into a hash grid.
Update Method	IUpdatable hierarchy	Each component manages its own per-frame update independently.
Observer (pull)	HUDComponent references	HUD holds const references to stat components and reads values at render time.
Object Pool	EnemyManager (SavageEnemy / ChomperEnemy pools), WorldItemPool	Pre-allocates a fixed array of entity slots, active flags control liveness. Eliminates per-spawn heap allocation and fragmentation during gameplay.
Observer (push variant)	DayTimerComponent.setOnExpired callback	On timer expiry, a stored std::function fires into Game without the component knowing what Game will do. Complements the pull-variant Observer already used in HUDComponent.

3.2 High-Level Architecture Diagram

High-Level System Architecture



4. Interface Design

Four pure-virtual interface classes form the foundation of the component system. Each interface is kept minimal, one responsibility per interface, following the Interface Segregation Principle.

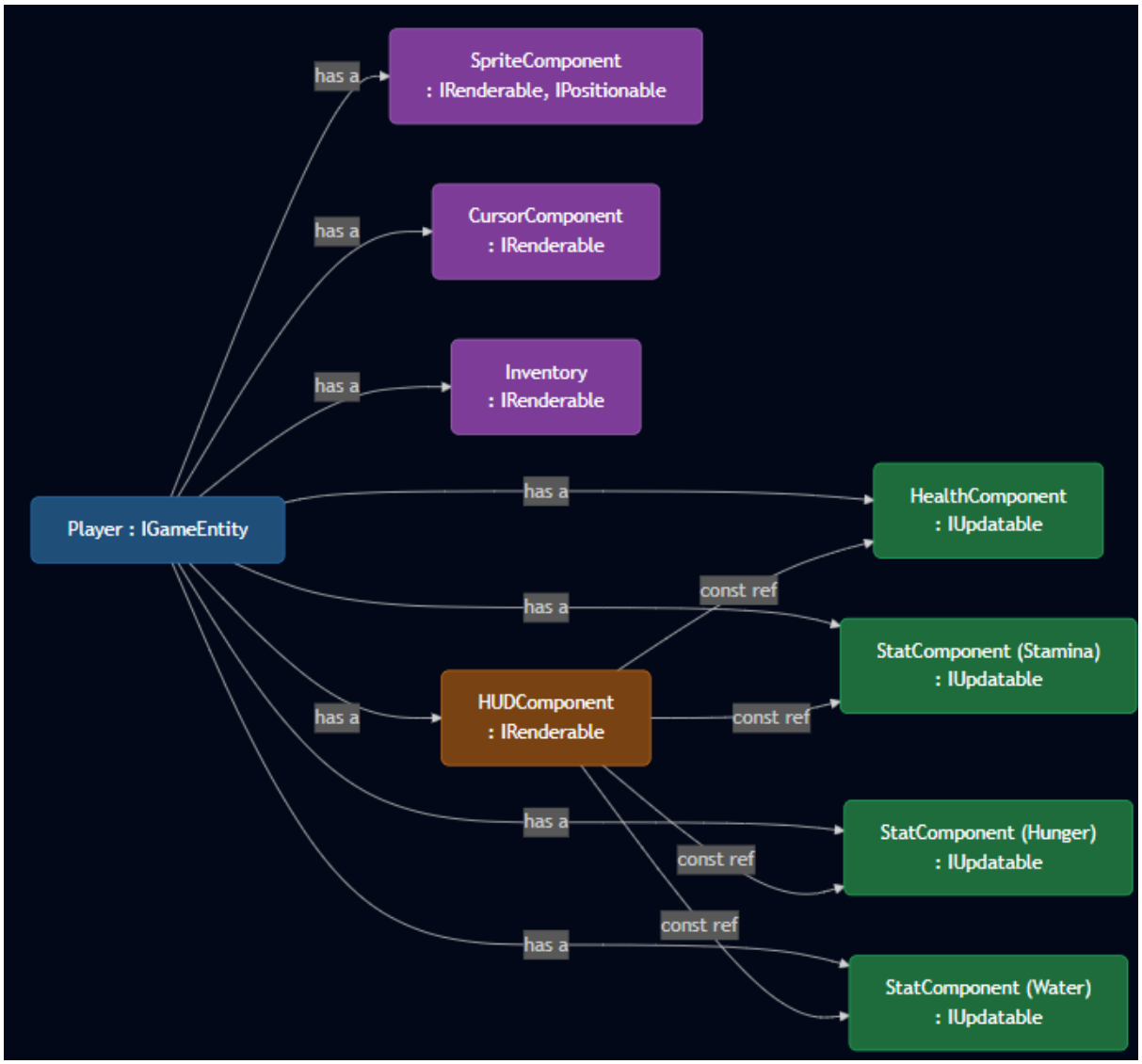
Interface	Key Method(s)	Purpose
IRenderable	render(RenderTarget&)	Any object that can be drawn to the screen.
IUpdatable	update(sf::Time)	Any object that requires per-frame logic updates.
IPositionable	getPosition() / setPosition()	Any object with a world-space or screen-space position.
ICollidable	getBounds()	Any object that participates in collision detection.
IGameEntity	isActive() / setActive()	Composite interface: inherits all four above. Used by Player and Enemy.

Design Note	IGameEntity inherits from all four interfaces. This is the one case where inheritance is justified, it is used as a convenience aggregate interface, not to share implementation. No method bodies are defined in IGameEntity itself.

5. Component Design

5.1 Component Overview

Player Component Composition



Declaration Order	HealthComponent and all three StatComponents must be declared before HUDComponent in Player.h. The HUDComponent constructor takes const references to these stats, if they are declared after HUDComponent, the reference will bind to an uninitialised object.

5.2 SpriteComponent

SpriteComponent wraps sf::Sprite and sf::Texture, handling texture loading, atlas region selection, scaling, rotation, and origin management. It implements both IRenderable and IPositionable.

Key responsibilities: texture loading from file path or shared reference, scale-based sizing (target width/height in pixels), centred origin support for correct rotation pivot, and bounds queries for AABB collision.

5.3 StatComponent

A generic parameterised stat class replacing three separate (Stamina, Hunger, Water) classes. Initialized with a maximum value and an optional drain rate (units per second). The update() method applies automatic drain each frame. The getRatio() method returns a normalised 0.0-1.0 value for HUD bar rendering.

5.4 HUDComponent

HUDComponent holds const references to HealthComponent and three StatComponents. At render time it reads their current values and ratios, this is the pull variant of the Observer pattern. The HUD never modifies stat state, it only reads it. Bars are rendered in screen space under m_uiView. Layout uses ScreenSize::s_width to avoid hardcoded magic numbers.

5.5 ButtonComponent

ButtonComponent implements a state machine with five visual states: Normal, Hovered, Pressed, Disabled, and Selected. The Selected state supports keyboard/gamepad navigation without mouse input. Callbacks are stored as std::function<void()> and executed on release (not press), preventing accidental double-triggers.

5.6 CursorComponent

A custom circular cursor replacing the OS cursor during gameplay. Its position is updated from screen-space mouse coordinates (InputController::getMousePosition()), not world-space. This is important, the player rotation uses a world-space mouse position obtained by mapping through the game view, while the cursor renders at the raw screen position.

5.7 DayTimerComponent

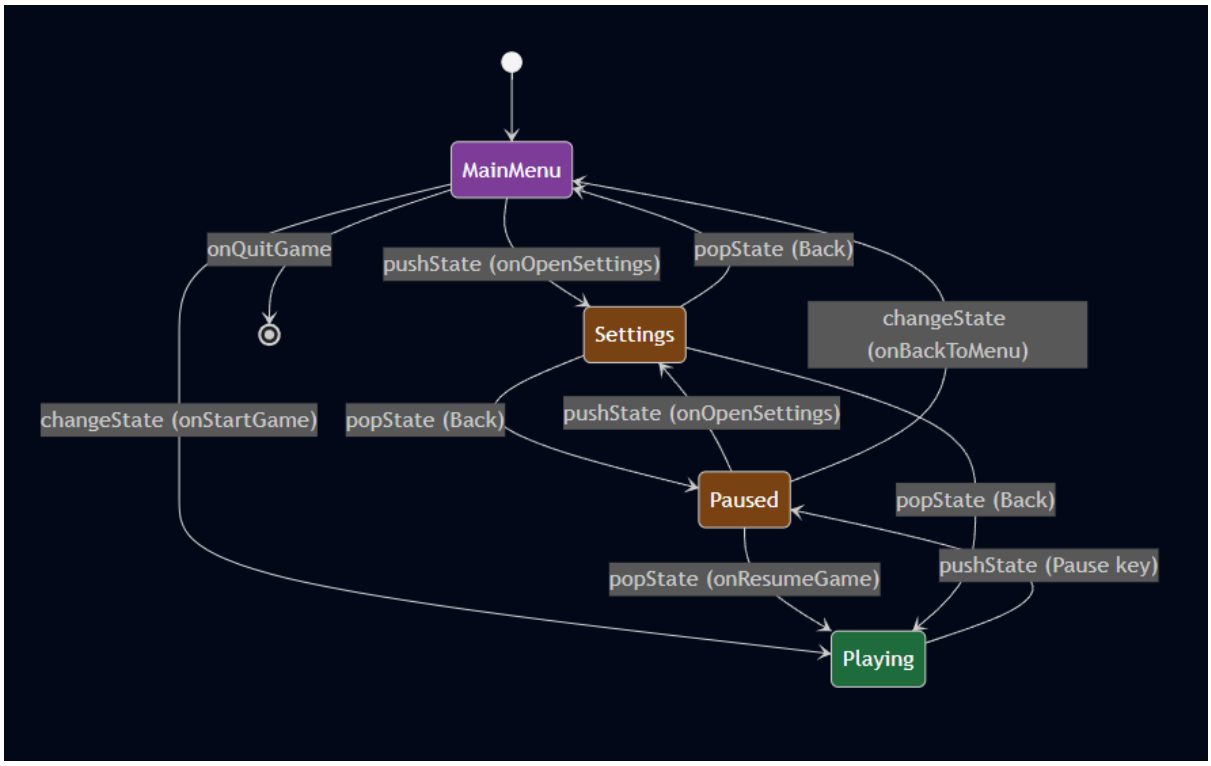
DayTimerComponent is a self-contained countdown timer with integrated HUD rendering and an expiry callback. It implements IUpdatable and IRenderable. The component is owned by Game and updated in updatePlaying, game regenerates upon timer expiry.

6. State Management: Pushdown Automaton

GameStateManager implements a pushdown automaton (PDA), a stack-based extension of the basic State pattern. Three transition operations are supported:

- pushState(newState): adds a state on top of the stack, preserving the current state beneath it. Used for pausing and settings overlays.
- popState(): returns to the previous state by popping the top. Used to resume gameplay or close settings.
- changeState(newState): clears the entire stack and sets a new root state. Used for major transitions such as returning to the main menu.

State Transition Diagram



State entry and exit callbacks (onStateEnter / onStateExit) allow Game.cpp to show or hide the appropriate UI, set cursor visibility, and perform cleanup without coupling the state manager to any specific game object.

7. Input System: Command Pattern

InputController abstracts keyboard and gamepad input using the Command pattern. Physical keys are not referenced directly in game logic, instead, named InputActions (MoveUp, Sprint, Inventory, Pause, etc.) are queried. Key bindings are stored in an std::unordered_map<InputAction, sf::Keyboard::Key> for O(1) lookup and runtime rebinding.

Current and previous frame states are stored in fixed-size bool arrays indexed by the enum integer value, avoiding heap allocation and giving O(1) access. memcpy and memset are used to copy and clear these arrays efficiently rather than per-element loops.

Three query types are provided:

- isPressed(action): true if the action is currently held.
- wasJustPressed(action): true only on the frame the action first became active. Used for one-shot triggers (inventory toggle, pause).
- wasJustReleased(action): true only on the frame the action was released. Used for button click detection.

8. Collision System

8.1 Overview

Collision detection uses a two-phase approach: AABB broad phase followed by SAT narrow phase for polygon shapes. The system is separated into CollisionManager (detection and resolution logic) and the ICollidable interface (getBounds contract). This separation allows any game entity to participate in collision without coupling to a specific entity type.

8.2 Collision Shape Types

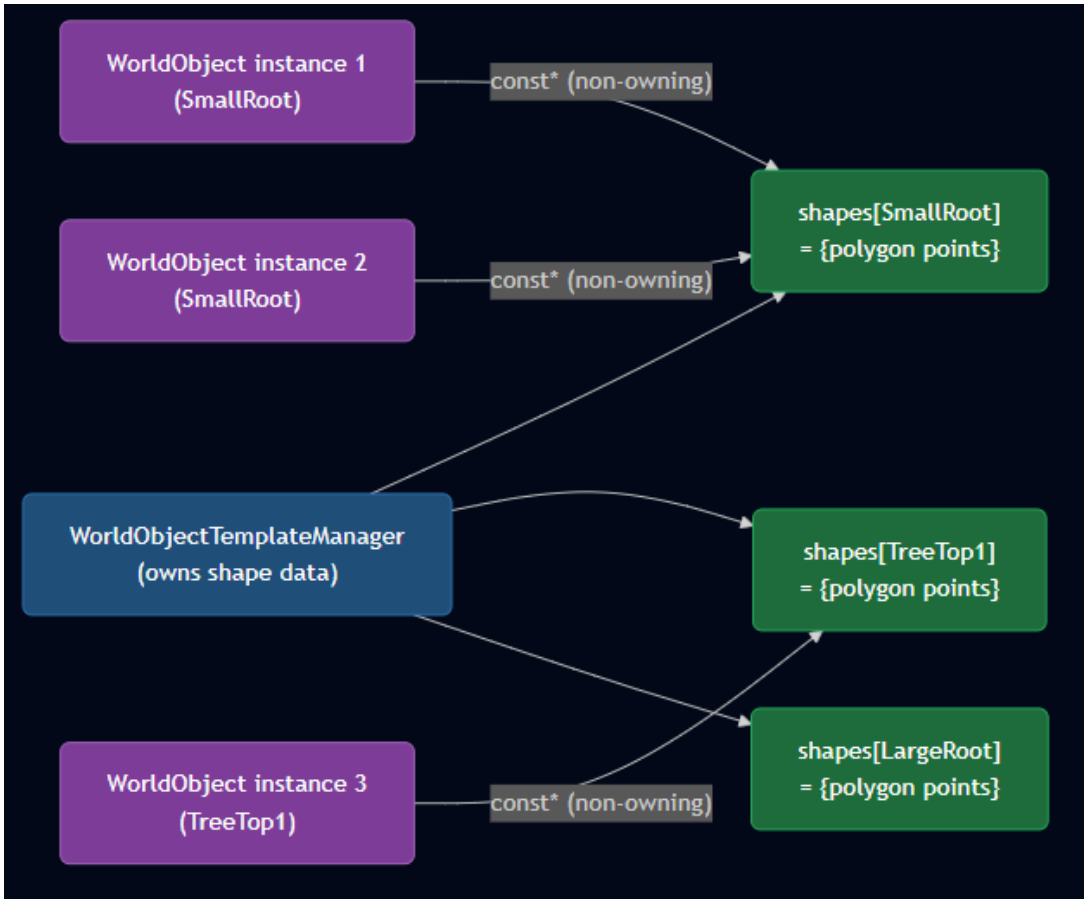
CollisionType.h defines a std::variant<sf::FloatRect, CollisionPolygon> as the CollisionShape type. This allows both axis-aligned rectangles and convex polygons to be stored in the same container and dispatched with std::visit without virtual calls.

Known Issue	CollisionType.h contains an anonymous struct with a points vector. This was a remnant of a planned circle collision type and is not currently in use. It can be safely removed in a future cleanup pass.

8.3 Flyweight Pattern: WorldObjectTemplateManager

Each WorldObject::Type has one CollisionShape list, loaded once from a TMX file by WorldObjectTemplateManager. Individual WorldObject instances hold a non-owning const pointer to the shared shape list plus a per-instance world-space offset. This avoids duplicating polygon data across potentially thousands of placed world objects.

Flyweight Collision Data Sharing



8.4 SAT Collision

The Separating Axis Theorem is applied when an AABB pre-check detects overlap. For each edge of the polygon, a perpendicular axis is derived. Both shapes are projected onto each axis and tested for overlap. If any axis shows separation, no collision exists. If all axes overlap, the minimum translation vector (MTV) is computed and used to resolve the penetration.

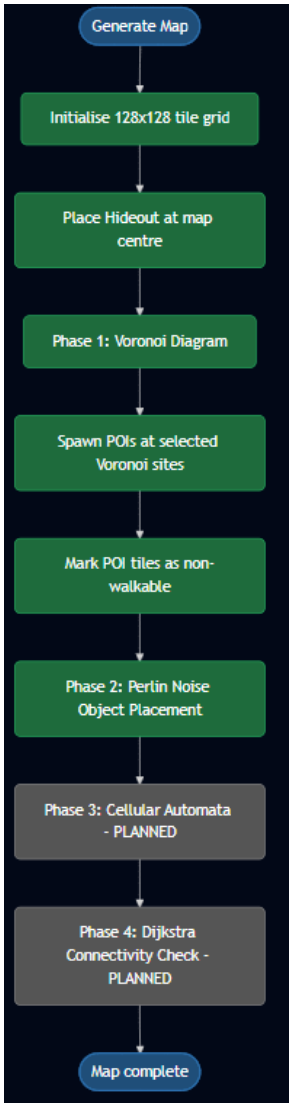
8.5 Coordinate Space

Polygon points are stored in sprite-local space. When queried for collision testing, getWorldSpaceShapes() translates each point by the object's world position and the per-type template origin offset. The template origin compensates for atlas pixel offsets, ensuring shapes align correctly with the rendered sprite at all scales.

9. Procedural Map Generation Pipeline

9.1 Overview

Generation Pipeline



9.2 Phase 1: Voronoi Diagram with Poisson Disk Sampling

Voronoi regionalization divides the map into organic cells. Each tile is assigned to its nearest site. Sites are generated using Robert Bridson's Poisson disk sampling algorithm, which guarantees a minimum distance between all sites while filling the space more uniformly than random rejection sampling.

Algorithm overview: an active list is seeded with one random point. Each iteration, a random active point is selected and up to 30 candidates are generated in the annular region between minDist and 2*minDist. Each candidate is checked against the spatial grid for proximity violations. Valid candidates are added to the active list and the site array, exhausted points are removed. The process continues until the active list is empty or the target site count is reached.

A SpatialGrid divides world space into cells sized at minDist. Each site is inserted into its cell. Proximity checks query only the 3x3 neighbourhood of cells around the candidate, giving O(1) average case instead of O(n) for naive rejection sampling.

9.3 Phase 2: Perlin Noise Object Placement

Ken Perlin's improved noise algorithm generates a continuous value field across the map. Octave noise (multiple noise layers at increasing frequency and decreasing amplitude) produces natural-looking density variation. Each tile position is sampled, tiles with noise values above a configurable threshold receive a world object. Object type is selected based on noise value range.

Object positions within each tile use Poisson disk sampling to prevent objects from clustering at tile boundaries. The minimum spacing between objects is a separate setting from the Voronoi site spacing.

9.4 Generation Settings

MapGenerator::GenerationSettings provides a clean configuration interface. Site count is configurable in two modes: ManualSites (explicit count) or AutoDensity (derived from a density preset: Sparse / Medium / Dense). This uses std::variant<ManualSites, AutoDensity> with std::visit for dispatch, avoiding a conditional branch in the generation code.

10. Rendering Pipeline

10.1 View Separation

Two SFML View objects are maintained throughout gameplay:

- m_gameView: follows the player in world space, clamped to map bounds. All terrain tiles, world objects, POIs, and the player sprite render under this view.
- m_uiView: fixed to screen resolution (1920x1080 default). All HUD elements, inventory panel, and the custom cursor render under this view.

Mixing these views causes hard-to-diagnose placement bugs. The render order in Game::render() is: world content under m_gameView -> HUD bars under m_uiView -> inventory panel under m_uiView -> cursor under m_uiView (last, always on top).

10.2 Frustum Culling

World objects are frustum-culled during rendering. The current camera view bounds are computed and only objects whose positions fall within the extended view rectangle are submitted to the render target. This reduces GPU draw calls significantly on large maps.

10.3 Shared Texture Atlas

All world objects of the same category share a single sf::Texture loaded once into memory. Individual SpriteComponent instances reference this shared texture and use setTextureRect() to select the appropriate atlas region. This eliminates per-object texture loads and GPU texture switches.

11. Enemy System: Object Pool and Manager

11.1 Pattern: Object Pool

EnemyManager implements the Object Pool pattern (Nystrom, 2014, ch. Object Pool). Two fixed-size std::array pools are declared:

```
std::array<SavageEnemy,  MAX_SAVAGE>  m_savagePool  // 8 slots
```

```
std::array<ChomperEnemy, MAX_CHOMPER> m_chomperPool  // 8 slots
```

All slots are initialised once in EnemyManager::initialize with their texture atlas loaded. setActive(false) marks each slot as dormant. spawnSavage and spawnChomper linearly scan for the first inactive slot, set its position, and activate it. despawnSavage and despawnChomper call setActive(false) without releasing memory. This gives O(n) spawn cost over the pool size, where n is at most 8, and O(1) despawn.

The Object Pool is appropriate here because enemy lifetimes are unpredictable and frequent heap allocation for short-lived entities would fragment memory. The fixed upper bound (MAX_SAVAGE, MAX_CHOMPER) is a deliberate design constraint: if the pool is exhausted the spawn call fails with a log message rather than allocating.

11.2 SavageEnemy

SavageEnemy implements IEnemy (which extends IGameEntity). Its AI runs a three-state machine (Idle, Chase, Lost) updated in updateAI, called from updateWithContext which receives deltaTime, player position, and map pointer. Line of sight checks via LineOfSight::hasLineOfSight gate state transitions. In Chase, velocity is set toward the player at CHASE_SPEED. In Lost, velocity is set toward the last known position at 60% of chase speed for LOST_DURATION seconds before returning to Idle.

takeDamage currently sets m_active = false (instant kill). A HealthComponent integration is noted as future work.

11.3 ChomperEnemy

ChomperEnemy extends the SavageEnemy AI with a fourth state: Leap. The Leap state is entered from Chase when the player is within LEAP_RANGE (220px) and the leap cooldown has elapsed. On entry, beginLeap locks a normalised direction vector toward the player and sets velocity to LEAP_SPEED (420px/s). The leap runs for LEAP_DURATION (0.35s) tracked by m_leapTimer, after which the enemy returns to Chase.

State	Entry Condition	Exit Condition	Velocity
Idle	Default / Lost timer expired	LOS established	Zero
Chase	LOS established	LOS lost / within leap range	CHASE_SPEED toward player
Leap	Within LEAP_RANGE, cooldown zero	LEAP_DURATION elapsed / wall hit	LEAP_SPEED locked direction
Lost	LOS lost from Chase	LOS re-established / LOST_DURATION	CHASE_SPEED * 0.5 toward last known

Collision during a Leap is handled via a dedicated applyCollisionCorrection method, which the EnemyManager calls separately from the standard resolveEnemyCollision template. A wall hit during Leap transitions the enemy to Chase and sets m_leapCooldownTimer to LEAP_COOLDOWN (3.0s).

12. Performance Analysis

12.1 Baseline: Unoptimized Generation

Initial Voronoi implementation used rejection sampling with O(n) distance checks against all existing sites. At 80x80 tile map size, total generation time was 84.2ms, already above a 1-frame budget at 60fps.

Benchmark	mapGeneration took 84.1777 ms | Map size: 80x80 | Method: rejection sampling, no spatial partition
	`====GENERATION TIME===`
	`mapGeneration took 84.1777 ms`
	`With a map size of 80x80`

12.2 Optimised: Spatial Partition + Poisson Disk Sampling

Replacing rejection sampling with Bridson's Poisson disk algorithm and adding the SpatialGrid hash-grid reduced generation time from 84ms to 11ms, a 7x improvement, at the same 80x80 map size.

Benchmark	mapMs Generation: 1ms | Voronoi Assignment: 10ms | Total: 11ms

```
=== Generate map inital performance ===
mapMs Generation: 1ms
Voronoi Assignment: 10ms
Total Generation: 11ms
====================
```

12.3 Regeneration Performance

Runtime map regeneration (R key) reuses the existing Map object rather than allocating a new one. Map::reset() clears all tile and POI data in place. Regeneration time is 10ms, comparable to initial generation, confirming no memory allocation overhead during regeneration.

Benchmark

Total Generation (regenerate): 10ms

```
=== Regenrate Map Performance ===
Total Generation: 10ms
====================
```

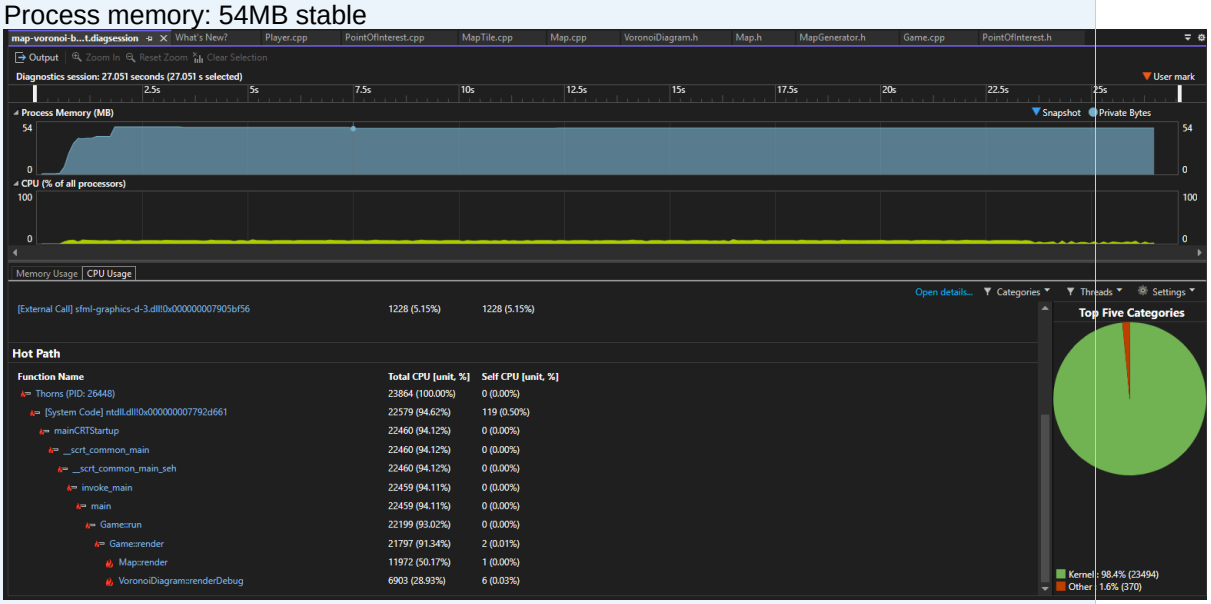
12.4 CPU Profiling: VS Diagnostics

Visual Studio CPU profiling over a 27 second session identified the hot path as Game::render() at 91.34% of total CPU, with Map::render() consuming 50.17% and VoronoiDiagram::renderDebug() at 28.93%. The game is render-bound, not logic-bound. The debug Voronoi boundary renderer is the primary performance bottleneck during development builds.

Process memory stabilised at approximately 54MB and remained flat throughout the session, confirming no memory leaks or unbounded allocation during gameplay.

Profiling Data

Game::render 91.34% | Map::render 50.17% | VoronoiDiagram::renderDebug 28.93% | Process memory: 54MB stable



12.5 Large Map Stress Test

A 128x128 tile map with full sprite building (16,384 sprites) and naive sprite construction reached 25,472ms generation time. This confirmed that per-tile sprite instantiation does not scale and motivated the move to a shared atlas and vertex array rendering approach.

Benchmark

mapGeneration took 25472.9ms | 16384 sprites created | World size: 8192x8192px

```
Tile sprites built: 16384 sprites created

--- Phase 2: Perlin Noise ---

--- Phase 3: Cellular Automata ---

--- Phase 4: Connectivity Check ---

=== Map Generation Complete ===

mapGeneration took 25472.9 ms
Map generated! World size: 8192x8192 pixels
TO BE REMOVED: Press 'R' during gameplay to regenerate map with new seed
Game initialized successfully!
```

13. Key Class Relationships

Map System Class Diagram

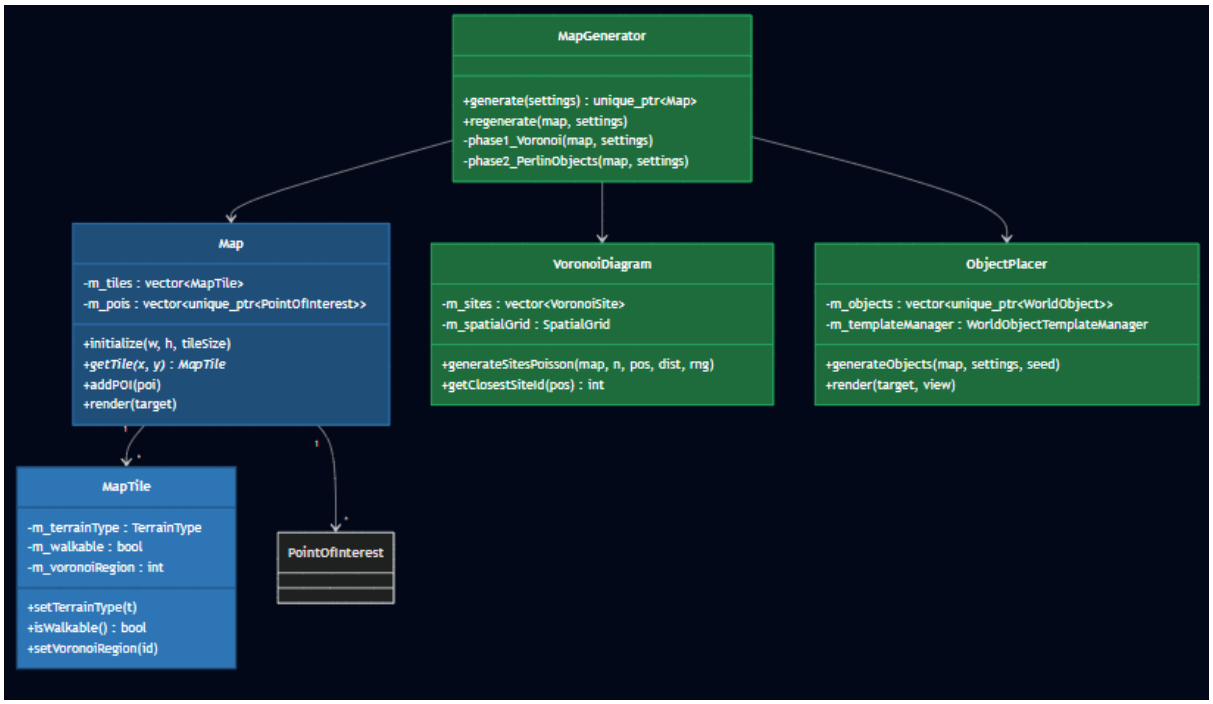
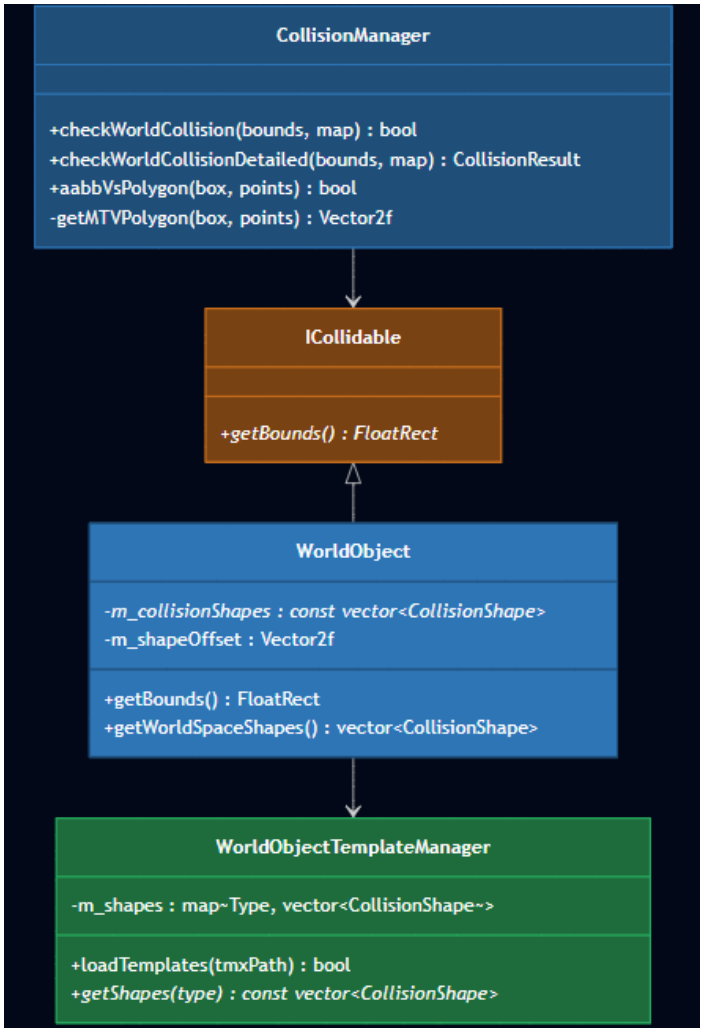


Diagram 7: Collision System Class Diagram



14. C++17 Features

Feature	Where Used	Rationale
std::variant / std::visit	CollisionShape, GenerationSettings::siteMode	Type-safe union for heterogeneous shape types and site mode selection, avoids virtual calls.
if constexpr	Template dispatch in components	Compile time branching for type-dependent logic without runtime overhead.
std::unique_ptr	Map, POIs, WorldObjects, Menus	RAII ownership, automatic cleanup with no manual delete calls.
Non-owning raw pointers	WorldObject::m_collisionShapes	Explicitly documents non-owning relationship, Flyweight pattern requires shared access without ownership.
const references	HUDComponent members	Zero-copy read access to stat

		data, documents non-modifying intent.
memcpy / memset	InputController state arrays	Cache friendly bulk state copy/clear on fixed size bool arrays, faster than element-wise loops.
structured bindings (planned)	Future result types	Improves readability of multi-value returns.

15. File and Module Structure

Module	Key Files	Responsibility
Core Loop	Game.h / Game.cpp	Main controller: event processing, state delegation, camera, render order.
State	GameStateManager.h/.cpp	Pushdown automaton stack for game state transitions.
Input	InputController.h/.cpp	Keyboard/gamepad abstraction, action queries, rebinding.
Player	Player.h/.cpp	Player entity: component aggregator + movement state machine.
Map	Map.h/.cpp, MapTile.h/.cpp	Tile grid container, POI management, coordinate utilities.
Generation	MapGenerator.h/.cpp, VoronoiDiagram.h/.cpp, PerlinNoise.h/.cpp, ObjectPlacer.h/.cpp	Multi-phase procedural generation pipeline.
Collision	CollisionManager.h/.cpp, CollisionType.h, WorldObjectTemplate.h/.cpp	SAT collision detection, shape types, Flyweight template manager.
UI/HUD	HudComponent.h/.cpp, Inventory.h/.cpp, ButtonComponent.h/.cpp, Menu.h/.cpp, SettingsMenu.h/.cpp	All UI rendering and interaction components.
POI	PointOfInterest.h/.cpp, POITemplate.h/.cpp, POITypeConfig.h/.cpp	POI data, TMX template loading, type configuration registry.
Utilities	MathUtilities.h, ScreenSize.h, ScreenSettings.h/.cpp, AssetPaths.h	Shared math, screen constants, resolution management, asset path constants.
Enemy System	EnemyManager.h/.cpp, SavageEnemy.h/.cpp, ChomperEnemy.h/.cpp	Object Pool for two enemy types, AI state machines, collision resolution, attack hit detection.
Day Timer	DayTimerComponent.h/.cpp	Countdown timer with HUD panel and push-variant Observer expiry

		callback.

References

[1] Nystrom, R. (2014). Game Programming Patterns. [Online]. Available at: https://gameprogrammingpatterns.com/

[2] Perlin, K. (2002). Improving Noise. In Proceedings of SIGGRAPH 2002. ACM Press. Available at: http://mrl.nyu.edu/~perlin/paper445.pdf

[3] Bridson, R. (2007). Fast Poisson Disk Sampling in Arbitrary Dimensions. ACM SIGGRAPH 2007 Sketches. Available at: https://dl.acm.org/doi/10.1145/1278780.1278807

[4] Solarian Programmer. (2012). Perlin Noise in C++11. [Online]. Available at: https://solarianprogrammer.com/2012/07/18/perlin-noise-cpp-11/

[5] leatherbee.org. (2018). Terrain Generation 3 Voronoi Diagrams. [Online]. Available at: https://leatherbee.org/index.php/2018/10/06/terrain-generation-3-voronoi-diagrams/

[6] GeeksforGeeks. Voronoi Diagram. [Online]. Available at: https://www.geeksforgeeks.org/dsa/voronoi-diagram/

[7] Cherno. (2022). std::unordered_map in C++. [Online]. Available at: https://youtu.be/KiB0vRi2wlc?si=c__NeniIIjXwTsD

[8] Pezzotti, J. (2020). Poisson Disk Sampling. [Online Video]. Available at: https://youtu.be/jofUe3Kjag4?si=HnIRv3t2Yv7CT_Gq

[9] SFML Development Team. (2024). SFML 3.0 Documentation. [Online]. Available at: https://www.sfml-dev.org/

[10] tmxlite Contributors. tmxlite Library. [Online]. Available at: https://github.com/fallahn/tmxlite